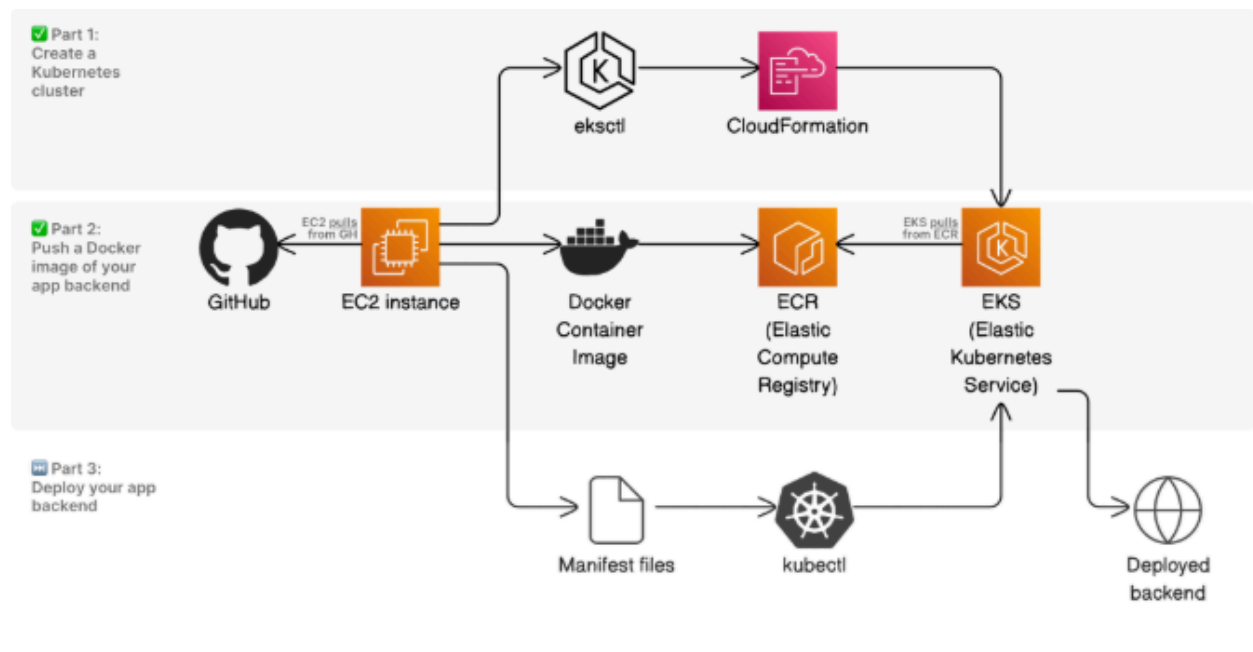


Application Infrastructure using Kubernetes Clusters



In this project, I am launching an application on Kubernetes Clusters using EKS

Launch and Connect to an EC2 Instance

- Head to the EC2 console.
- Check that you're using the **AWS Region** that's closest to you.
- Click **Launch instances**.
- Name your EC2 instance nextwork-eks-instance
- For the **Amazon Machine Image**, select **Amazon Linux 2023 AMI**
- For the Instance type, select **t3.micro**.
- For the Key pair, Select **Proceed without a key pair (not recommended)**.
- In the Network settings section, we'll keep the **default security group for now**
- **Select Launch instance.**
- In your **EC2 console's Instances** page, select your new instance.
- Select **Connect**. Welcome to EC2 Instance Connect! EC2 Instance Connect is a tool that makes it easy to connect to your instance directly from your browser.
- Select **Connect** again in your EC2 Instance Connect page
-

Install eksctl

Run the following command in your EC2 instance's terminal to download, extract, and install **eksctl**.

```
curl --silent --location
"https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_$(uname
-s)_amd64.tar.gz" | tar xz -C /tmp
sudo mv -v /tmp/eksctl /usr/local/bin
```

Verify that you've installed eksctl:

```
eksctl version
```

You should spot a version number in the terminal output.

Set up IAM role for your EC2 Instance

Let's create a new IAM role for your EC2 instance! If you've done this before in the [first project of this series](#), you might still have it in your AWS account.

- Head to the IAM console.
- Select **Roles**.
- Select **Create role**.
- Under **Trusted entity type**, select **AWS service** to tell AWS that we're setting up this role for a AWS service (Amazon EC2).
- Under **Use case**, select **EC2**.
- Select **Next**.
- Under **Permissions policies**, we'll grant our EC2 instance **AdministratorAccess**.
- Make sure the **AdministratorAccess** option is checked, and select **Next**.
- Let's give this role a straightforward name - nextwork-eks-instance-role

- Enter a short description: Grants an EC2 instance AdministratorAccess to my AWS account. Created during NextWork's Kubernetes project.
- **Create Role**

Trusted entity type

☒ **AWS service**
Allow AWS services like EC2, Lambda, or others to perform actions in this account.

☐ **AWS account**
Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.

☐ **Web identity**
Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.

☐ **SAML 2.0 federation**
Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.

☐ **Custom trust policy**
Create a custom trust policy to enable others to perform actions in this account.

Use case
Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case

EC2 ▼

Choose a use case for the specified service.

Use case

☒ **EC2**
Allows EC2 instances to call AWS services on your behalf.

Attach IAM role to EC2 instance

- Head back to the **Amazon EC2** console.
- Select **Instances** from the left hand sidebar.
- Select the checkbox next to your **nextwork-eks-instance** EC2 instance.
- Select the **Actions** dropdown, and then **Security -> Modify IAM role**.
- Under **IAM role**, select your new **nextwork-eks-instance-role** role.
- Select **Update IAM role**

Instance ID
i-030d49d1d59d07a56 (nextwork-eks-instance)

IAM role
Select an IAM role to attach to your instance or create a new role if you haven't created any. The role you select replaces any roles that are currently attached to your instance.

nextwork-eks-instance-role ▼

 [Create new IAM role](#)

[Cancel](#) [Update IAM role](#)

Create EKS Cluster

- Back to **EC2 Instance Connect**, run the following command to create your EKS cluster.
- Make sure to replace your-region-code with your AWS region's code e.g. us-west-2:

```
eksctl create cluster \
  --name nextwork-eks-cluster \
  --nodegroup-name nextwork-nodegroup \
  --node-type t3.micro \
  --nodes 3 \
  --nodes-min 1 \
  --nodes-max 3 \
  --version 1.33 \
  --region us-east-2
```

Creating your EKS cluster can take some time (around 15-20 minutes), in the meantime...

Pull Code from Github Repository

- While your EC2 instance is busy with creating your cluster, head back to the **EC2** console.
- Select your EC2 instance **nextwork-eks-instance-role**, and select **Connect** again.
- Start a new **EC2 Instance Connect** session with your instance.
- In your new **EC2 Instance Connect** tab, run the command to install Git:

```
sudo dnf update
```

```
sudo dnf install git -y
```

Verify you've downloaded Git by checking for its version:

```
git --version
```

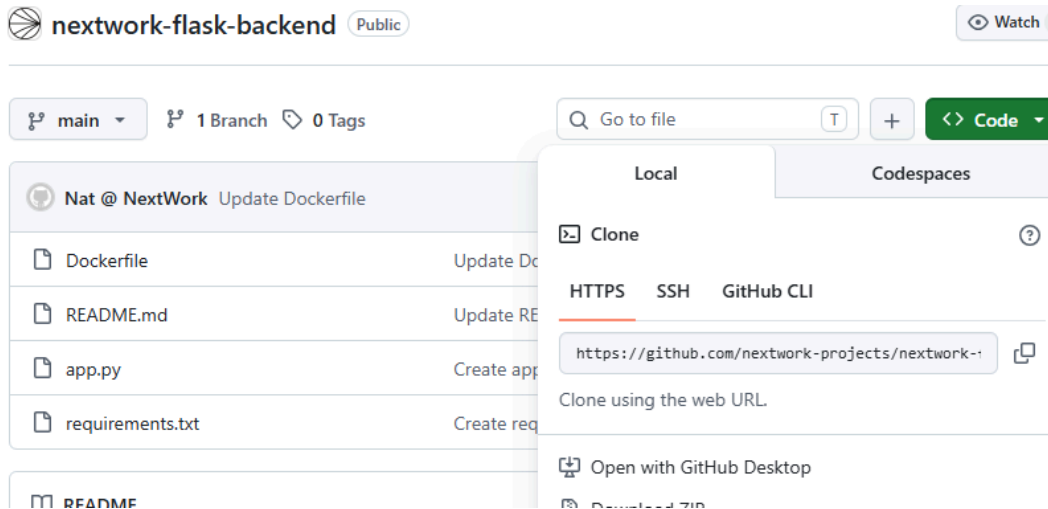
Configure Git by running the command below. Make sure to replace the placeholder values with your name and email:

```
git config --global user.name "Russell Geisler"
```

```
git config --global user.email "russellsgeisler@gmail.com"
```

In a new tab, visit your [team member's GitHub repository](#)

- Select **Code** to reveal the instructions for cloning their repository.
- Copy the **HTTPS** URL.



- Head back to your **EC2 Instance Connect** tab.
- In the terminal, run the command to clone your team member's repository. Replace your-github-url with the HTTPS URL you've copied:

```
git clone your-github-url
```

- Run **ls**, which is the Linux command to list all files and subdirectories in your current directory.
- Confirm that you have a new folder called **nextwork-flask-backend** in your EC2 instance.

Nice work, seeing the folder means you've successfully cloned your team member's backend code!

- Navigate into the application directory and run the command that lists everything inside:

```
cd nextwork-flask-backend
```

```
ls
```

```

Installed:
  git-2.50.1-1.amzn2023.0.1.x86_64      git-core-2.50.1-1.amzn2023.0.1.x86_64  git-core-doc-2.50.1-1.amzn2023.0.1.x86_64
  perl-File-Find-1.37-477.amzn2023.0.7.noarch  perl-Git-2.50.1-1.amzn2023.0.1.noarch  perl-TermReadKey-2.50.1-1.amzn2023.0.1.noarch

Complete!
[ec2-user@ip-172-31-9-157 ~]$ git config --global user.name "Russell Geisler"
-bash: $'\E[200~git': command not found
[ec2-user@ip-172-31-9-157 ~]$ git config --global user.email "russellsgeisler@gmail.com"
[ec2-user@ip-172-31-9-157 ~]$ git config --global user.name "Russell Geisler"
[ec2-user@ip-172-31-9-157 ~]$ git config --global user.email "russellsgeisler@gmail.com"
[ec2-user@ip-172-31-9-157 ~]$ git clone https://github.com/nextwork-projects/nextwork-flask-backend.git
Cloning into 'nextwork-flask-backend'...
remote: Enumerating objects: 18, done.
remote: Counting objects: 100% (18/18), done.
remote: Compressing objects: 100% (17/17), done.
remote: Total 18 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (18/18), 6.14 KiB | 6.14 MiB/s, done.
Resolving deltas: 100% (4/4), done.
[ec2-user@ip-172-31-9-157 ~]$ ls
nextwork-flask-backend
[ec2-user@ip-172-31-9-157 ~]$ cd nextwork-flask-backend
ls
Dockerfile  README.md  app.py  requirements.txt
[ec2-user@ip-172-31-9-157 nextwork-flask-backend]$

```

Install and Configure Docker

- Install Docker:

sudo yum install -y docker

Then

sudo service docker start

Give User Access to Docker

- To confirm which user you're using in the terminal, run the command `whoami`
- You should see **ec2-user** in the terminal response.
- Add **ec2-user** to the **Docker** group:

sudo usermod -a -G docker ec2-user

- Restart your **EC2 Instance Connect** session by refreshing your current tab.
- Make sure your `ec2-user` has been added to the Docker group:

groups ec2-user

- We'll jump back into the application directory. We'll also run the `ls` command again to list everything inside, so we can double check we're at the right place:

cd nextwork-flask-backend

/s

- Run the command for building your Docker image:

Sudo docker build -t nextwork-flask-backend .

Push container image to AWS ECR

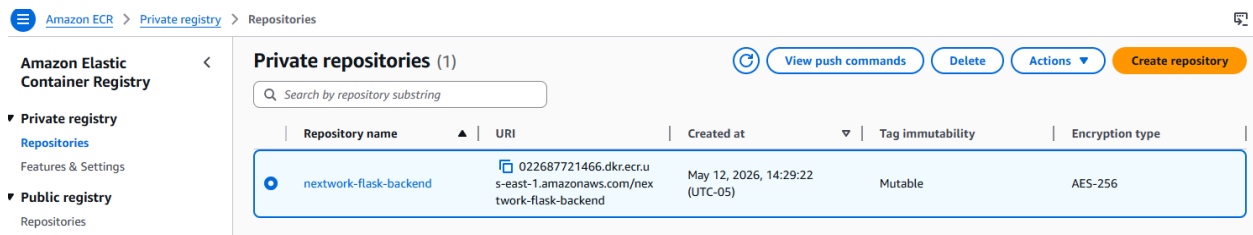
- Create a new Amazon ECR repository using this command:

*aws ecr create-repository *

*--repository-name nextwork-flask-backend *

*--image-scanning-configuration scanOnPush=true *

- In a new tab, head to the ECR console.
- Confirm that you can see a new repository called **nextwork-flask-backend**



Our Amazon ECR repository is live. We're ready to push our container image into ECR!

Push your container image to ECR

- Select your new repository.
- Select **View push commands**.
- Copy the third, fourth, then first commands.
- Run the command in your EC2 Instance Connect window.

Push commands for nextwork-flask-backend



macOS / Linux

Windows

Make sure that you have the latest version of the AWS CLI and Docker installed. For more information, see [Getting Started with Amazon ECR](#).

Use the following steps to authenticate and push an image to your repository. For additional registry authentication methods, including the Amazon ECR credential helper, see [Registry Authentication](#).

1. Retrieve an authentication token and authenticate your Docker client to your registry. Use the AWS CLI:

```
aws ecr get-login-password --region us-east-2 | docker login --username AWS --password-stdin 022687721466.dkr.ecr.us-east-2.amazonaws.com
```

Note: If you receive an error using the AWS CLI, make sure that you have the latest version of the AWS CLI and Docker installed.

2. Build your Docker image using the following command. For information on building a Docker file from scratch see the instructions [here](#). You can skip this step if your image is already built:

```
docker build -t nextwork-flask-backend .
```

3. After the build completes, tag your image so you can push the image to this repository:

```
docker tag nextwork-flask-backend:latest 022687721466.dkr.ecr.us-east-2.amazonaws.com/nextwork-flask-backend:latest
```

4. Run the following command to push this image to your newly created AWS repository:

```
docker push 022687721466.dkr.ecr.us-east-2.amazonaws.com/nextwork-flask-backend:latest
```

- Head back to your ECR console's push commands window.
- We'll skip the second command - we've already built our container image!
- Copy the last two push commands
- Run the last two commands in your EC2 Instance Connect terminal to tag and push your container image

```
Login Succeeded
docker push 022687721466.dkr.ecr.us-east-2.amazonaws.com/nextwork-flask-backend:latestlogin --username AWS --password-stdin 022687721466.dkr.ecr.us-east-2.amazonaws.com/nextwork-flask-backend
The push refers to repository [022687721466.dkr.ecr.us-east-2.amazonaws.com/nextwork-flask-backend]
c9ffde42b4e5: Pushed
c1e3ac9884ab: Pushed
6be2d6f7e5ee: Pushed
4fdeb77ab0d4: Pushed
4db33e6b6fc0: Pushed
a1bc7c8917ec: Pushed
695da10e684a: Pushed
256f393e029f: Pushed
latest: digest: sha256:cfb2e72729ce603904fc021fe656db203be47105ff249406db55274c33eb7c80 size: 1991
[ec2-user@ip-172-31-9-157 ~]$
```

- Head back to the **ECR** console.
- Close the push commands window.
- Select the refresh button to refresh your console.

Setting up the app for deployment

- Head back into your EC2 Instance Connect tab.

- Create a new directory from your instance's root called manifests. Run the following commands:

```
cd ..
```

```
mkdir manifests
```

```
cd manifests
```

- Create the Deployment manifest file by running the following command:

```
cat << EOF > flask-deployment.yaml
```

```
---
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: nextwork-flask-backend
```

```
  namespace: default
```

```
spec:
```

```
  replicas: 3
```

```
  selector:
```

```
    matchLabels:
```

```
      app: nextwork-flask-backend
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: nextwork-flask-backend
```

```
    spec:
```

```
      containers:
```

```
        - name: nextwork-flask-backend
```

image:
022687721466.dkr.ecr.us-east-2.amazonaws.com/nextwork-flask-backend

ports:

- containerPort: 8080

EOF

- Run *nano flask-deployment.yaml* in your terminal to see your work.

```
GNU nano 8.3 flask-deployment.yaml
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nextwork-flask-backend
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nextwork-flask-backend
  template:
    metadata:
      labels:
        app: nextwork-flask-backend
    spec:
      containers:
        - name: nextwork-flask-backend
          image: 022687721466.dkr.ecr.us-east-2.amazonaws.com/nextwork-flask-backend
          ports:
            - containerPort: 8080

[ Read 21 lines ]
^G Help      ^O Write Out  ^S Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

Create your Service Manifest

Create the flask-service.yaml file:

cat << EOF > flask-service.yaml

apiVersion: v1

kind: Service

metadata:

name: nextwork-flask-backend

spec:

selector:

app: nextwork-flask-backend

type: NodePort

ports:

- port: 8080

targetPort: 8080

protocol: TCP

EOF

Deploy the backend application

- Install kubectl:

```
sudo curl -o /usr/local/bin/kubectl \
```

```
https://s3.us-west-2.amazonaws.com/amazon-eks/1.31.0/2024-09-12/bin/linux/amd64/k  
ubectl
```

- Give yourself permissions

```
sudo chmod +x /usr/local/bin/kubectl
```

```
sudo aws eks update-kubeconfig --name nextwork-eks-cluster --region us-east-2
```

- Then apply yaml files

```
kubectl apply -f flask-deployment.yaml
```

```
kubectl apply -f flask-service.yaml
```

The files were successfully updated and the cluster and application are live